

ArrayList in Java

An **ArrayList** in Java is part of the `java.util` package and is a resizable array implementation of the **List** interface. Unlike arrays, which have a fixed size, an `ArrayList` can grow or shrink dynamically as elements are added or removed.

Key Features of ArrayList:

1. **Resizable:** The size of an `ArrayList` can grow or shrink dynamically as elements are added or removed.
2. **Ordered:** Elements in an `ArrayList` are ordered based on the sequence in which they are inserted.
3. **Index-based Access:** Elements can be accessed using an index, similar to arrays.
4. **Allows Duplicates:** An `ArrayList` can store duplicate elements.
5. **Null Elements:** It can contain `null` elements (if allowed by the use case).

Declaration and Initialization

To work with an `ArrayList`, you need to import it from the `java.util` package.

1. Declaration:

```
// Declare an ArrayList of integers
ArrayList<Integer> list;
```

2. Initialization:

```
// Initialize an ArrayList of integers with some initial capacity
ArrayList<Integer> list = new ArrayList<>();
```

3. Initialize with Specific Values:

```
// Initialize an ArrayList with specific values
ArrayList<String> fruits = new ArrayList<>(Arrays.asList("Apple", "Banana", "Mango"));
```

Adding Elements to ArrayList

You can add elements to an `ArrayList` using the `add()` method.

Example:

```

public class ArrayListExample {
    public static void main(String[] args) {
        // Create an ArrayList of integers
        ArrayList<Integer> numbers = new ArrayList<>();

        // Adding elements
        numbers.add(10); // Adds 10 to the list
        numbers.add(20); // Adds 20 to the list
        numbers.add(30); // Adds 30 to the list

        // Output the ArrayList
        System.out.println("ArrayList: " + numbers);
    }
}

```

- **Output:**

```
ArrayList: [10, 20, 30]
```

Accessing Elements

You can access elements in an `ArrayList` by using the `get()` method and specifying the index.

Example:

```

public class AccessArrayList {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");

        // Accessing elements
        System.out.println("First fruit: " + fruits.get(0)); // Output:
Apple
        System.out.println("Second fruit: " + fruits.get(1)); // Output:
Banana
    }
}

```

- **Output:**

```
First fruit: Apple
```

```
Second fruit: Banana
```

Modifying Elements

You can modify elements in an `ArrayList` using the `set()` method, which takes an index and a new value.

Example:

```
public class ModifyArrayList {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");

        // Modifying the second element
        fruits.set(1, "Mango");

        // Output the modified ArrayList
        System.out.println("Modified ArrayList: " + fruits);
    }
}
```

- **Output:**

```
Modified ArrayList: [Apple, Mango, Orange]
```

Removing Elements

You can remove elements from an `ArrayList` using the `remove()` method, either by specifying the index or the element itself.

Example:

```
public class RemoveFromArrayList {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");

        // Remove by index
        fruits.remove(1); // Removes the element at index 1 (Banana)
    }
}
```

```

        // Remove by value
        fruits.remove("Orange"); // Removes "Orange" from the list

        // Output the remaining ArrayList
        System.out.println("ArrayList after removal: " + fruits);
    }
}

```

- **Output:**

```
ArrayList after removal: [Apple]
```

Size of ArrayList

You can get the number of elements in an `ArrayList` using the `size()` method.

Example:

```

public class ArrayListSize {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");

        // Get the size of the ArrayList
        System.out.println("Size of ArrayList: " + fruits.size());
    }
}

```

- **Output:**

```
Size of ArrayList: 3
```

Checking if an Element Exists

You can check if an element exists in an `ArrayList` using the `contains()` method, which returns a boolean value.

Example:

```

public class CheckElementInArrayList {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");

        // Check if the ArrayList contains "Banana"
        if (fruits.contains("Banana")) {
            System.out.println("Banana is present in the list");
        } else {
            System.out.println("Banana is not present in the list");
        }
    }
}

```

- **Output:**

```
Banana is present in the list
```

Example of a Complete Program Using ArrayList

```

import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        // Create an ArrayList of integers
        ArrayList<Integer> numbers = new ArrayList<>();

        // Add elements
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        // Print the ArrayList
        System.out.println("Numbers: " + numbers);

        // Modify an element
        numbers.set(1, 25); // Change 20 to 25
        System.out.println("Modified Numbers: " + numbers);

        // Remove an element
        numbers.remove(Integer.valueOf(25)); // Removes the value 25
        System.out.println("After Removal: " + numbers);

        // Get the size of the ArrayList
    }
}

```

```
        System.out.println("Size of the ArrayList: " + numbers.size());  
    }  
}
```

- **Output:**

```
Numbers: [10, 20, 30]  
Modified Numbers: [10, 25, 30]  
After Removal: [10, 30]  
Size of the ArrayList: 2
```